

Global Health TRE - MVP model and architecture

o. Pain-point scope and traceability

Product: Global Health Trusted Research Environment (TRE) **Course:** Advanced Software Engineering - Model and Architect Your MVP **Team:** Anu, Ade, Manushi, Jibek

The MVP has two sides. Dr. Amina Kamau uploads a file she already has and records what local terms mean. The TRE drafts an analysis file and can suggest translations for Dr. Ezer Kang. Daniel Okoro decides whether that researcher may receive the analysis file, and the TRE stores the decision. Later diagrams trace to a row in the table below.

Personas in this MVP

Persona	Role in the MVP
Dr. Ezer Kang	Medical researcher at Howard University. He accepts an analysis-ready file and accepted translations. He receives that file only after a custodian approves his request.
Dr. Amina Kamau	Physician and clinical research coordinator at a regional hospital in Kenya. She uploads study data without a separate cleaning project and records local terms.
Daniel Okoro	National health-data custodian. He approves or denies access and can see who asked, why, and what was decided.

Marcus Lee and Bayowa O. are not drivers of this scope. Bayowa's hospital records and board reports are out of scope. Comparing many external datasets inside the TRE is also out of scope.

Traceability table

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
PP-1	Dr. Ezer Kang	After a file is exported, he still reformats	The TRE drafts a data dictionary	UC-4 Prepare analysis file	Every uploaded column has a

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
		rows by hand and writes the data dictionary himself before SPSS or R can use it.	(name, label, type) and a rectangular analysis file from the uploaded table. He edits names and accepts the draft.		dictionary row. The accepted file uses those names. He does not build the dictionary by hand on the happy path.
PP-2	Dr. Amina Kamau	Study data sits in spreadsheets, notes, and exports. She has no time to clean it, so it is never shared for research.	She uploads a spreadsheet or CSV she already has. The TRE checks missing values and basic field shape and stores the result as a dataset.	UC-1 Upload dataset; UC-2 Check dataset	She submits an uncleaned file and gets a check report in one sitting. Her active work on that file stays under 15 minutes.
PP-3	Dr. Amina Kamau	Local terms and abbreviations differ by site. A researcher in another country can misread them.	She marks a field or value as a local term and writes what it means. The definition is stored with the dataset and shown with the released analysis file.	UC-3 Document local term	A term she flags cannot be published without a definition. The released analysis file shows that definition beside the field.
PP-4	Daniel Okoro	Sensitive health data can be reused without a clear record of who is using it, why, and how the contributing population benefits.	Every access request names the dataset, the purpose, and the expected benefit. He approves or denies it. The TRE stores his identity and the time. Approval releases the accepted analysis file only. The	UC-6 Request dataset access; UC-7 Review access request	For any dataset he can list requester, purpose, benefit, decision, and timestamp. A request with no purpose cannot be stored.

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
			source upload stays in storage.		
PP-5	Dr. Ezer Kang	Translating questionnaires and field labels into another language takes significant time.	The TRE suggests a translation of an instrument item or dictionary label. The suggestion is kept only after he edits or accepts it.	UC-5 Translate study text	Each accepted item stores the source text, the accepted translation, the language, and who accepted it. Unreviewed machine text is not published.

Reserved use cases, reused with these same names in every later diagram:

ID	Name	Pain point	Relationship
UC-1	Upload dataset	PP-2	Base use case for intake
UC-2	Check dataset	PP-2	Included by UC-1. The check always runs.
UC-3	Document local term	PP-3	Extends UC-1 when the file has a local term.
UC-4	Prepare analysis file	PP-1	Researcher accepts the drafted file.
UC-5	Translate study text	PP-5	Researcher accepts or edits a suggestion.
UC-6	Request dataset access	PP-4	Researcher submits purpose and benefit.
UC-7	Review access request	PP-4	Custodian approves or denies.

Out of scope

The MVP does not include:

- Finding datasets by search, comparing populations, or analyzing data in a notebook inside the TRE.
- Cultural adaptation of instruments: redesigning answer scales, back-translation as a formal method, or testing whether a translation still measures the same concept.
- Qualitative coding of interview transcripts.
- A hospital operations system: wards, emergency records, paper-chart migration, and reports for a hospital board or a ministry.

- Building KoboToolbox, SPSS, R, or a hospital record system. The researcher uses the released analysis file in the tools he already has.
- Sharing a dataset by email or by any path that skips the access request.
- Automatic approval. A custodian makes the decision.

Boundary this scope assumes

We build dataset intake, the check report, the data dictionary, local-term definitions, translation review, access requests, and the approval record. We integrate sign-in, file storage, machine translation, and notification. We do not build those four. The context diagram (A1) is the picture of that split.

Walkthrough thread

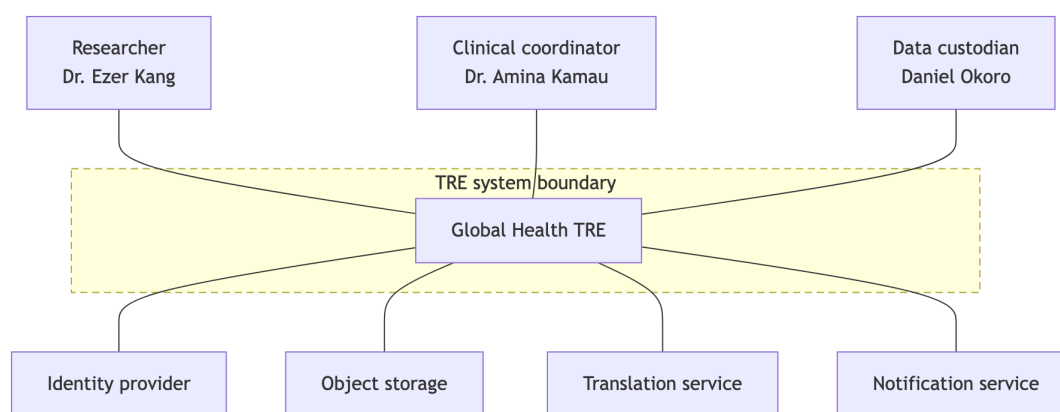
The in-class walkthrough follows **PP-4**. Dr. Kang requests access to a published dataset (UC-6). Daniel Okoro reviews that request (UC-7). PP-1, PP-2, and PP-3 are what made the dataset publishable. PP-5 is in the MVP and is not the walkthrough path.

Architecture constraint

The architecture decision record uses a **modular monolith**: one deployable application with modules for intake, dictionary, translation review, and access. The Kubernetes manifest deploys that one application.

A1. Context diagram

Caption: This context diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5 by showing the TRE as one system between the three personas and the external services it uses.



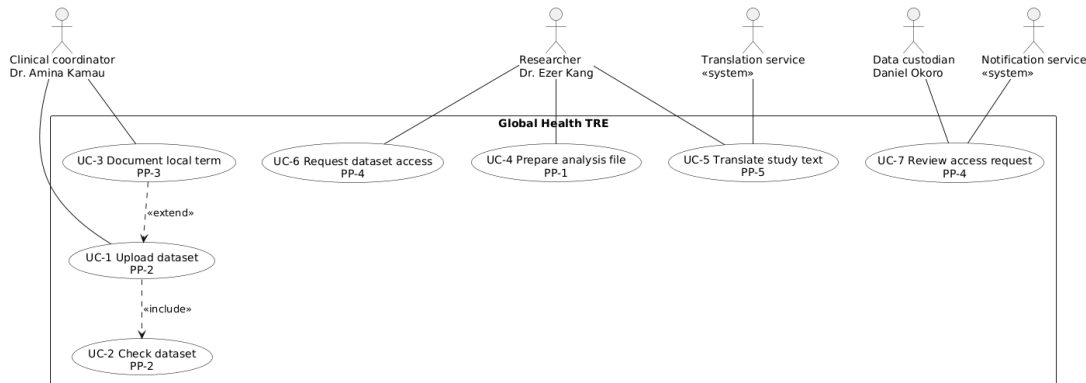
Context diagram

Boundary justification

The TRE builds the approval record: who asked for a dataset, why, and what Daniel Okoro decided. It also builds the dictionary, the local-term definitions, and the step where Dr. Kang accepts a translation. Sign-in and suggested translations are integrated. The identity provider owns accounts. The translation service owns the language model. The TRE only stores the text Dr. Kang has accepted.

A2. Use case diagram

Caption: This use case diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5. Each use case is tagged with the pain point it addresses.



Use case diagram

The exported figure is drawn from A2-use-cases.puml so the use cases sit inside the system boundary as ovals. The Mermaid block above is the same model: the same actors, the same seven use cases, the same include, and the same extend.

Human actors are the three personas. Translation service and Notification service are system actors. They also appear on the context diagram.

The **include** arrow points from UC-1 Upload dataset to UC-2 Check dataset. Every upload runs the check. The **extend** arrow points from UC-3 Document local term back to UC-1 Upload dataset. Documenting a term is extra behavior, used when the file contains a local term, and it is not required for every upload.

A3. Textual use cases

The two use cases below are the walkthrough for PP-4. The use case diagram still carries PP-1, PP-2, PP-3, and PP-5.

UC-6 Request dataset access

Caption: This use case supports PP-4. It creates the record of who wants a dataset and why.

Row	Detail
Actors	Researcher (Dr. Ezer Kang). Identity provider. TRE, through AccessRequest, Dataset, and AuditRecord.
Description	The researcher asks to receive the accepted analysis file for one published dataset. He must state the purpose and the expected benefit to the people the data came from.
Data	Session, datasetId, purpose, benefit.
Stimulus	The researcher submits the access-request form.
Response	The TRE returns a requestId. The request status is Submitted. An audit record stores the action submitted.
Pre/post-conditions	Pre: the researcher has a session, and the dataset exists. Post: an AccessRequest is Submitted, with purpose and benefit stored. If an alternative flow runs, no Submitted request is kept.
Comments	Supports PP-4. This is the first half of the class walkthrough. The source upload is not sent to the researcher at this step.
Alternative flows	1. Purpose is empty, or Dataset.isPublished() is false: AccessRequest returns rejected and writes no submitted audit record. 2. The researcher calls withdraw() before review: AuditRecord.record(withdrawn) runs, and the status becomes Withdrawn.

UC-7 Review access request

Caption: This use case supports PP-4. It stores the custodian decision and tells the researcher.

Row	Detail
Actors	Data custodian (Daniel Okoro). TRE, through AccessRequest and AuditRecord. Notification service.
Description	The custodian reads a submitted request and approves or denies it. Approval releases the accepted analysis file to that researcher. The source file stays in object storage.
Data	requestId, decision, deny reason, custodian identity, decidedAt.
Stimulus	The custodian opens a submitted request and chooses approve or deny.

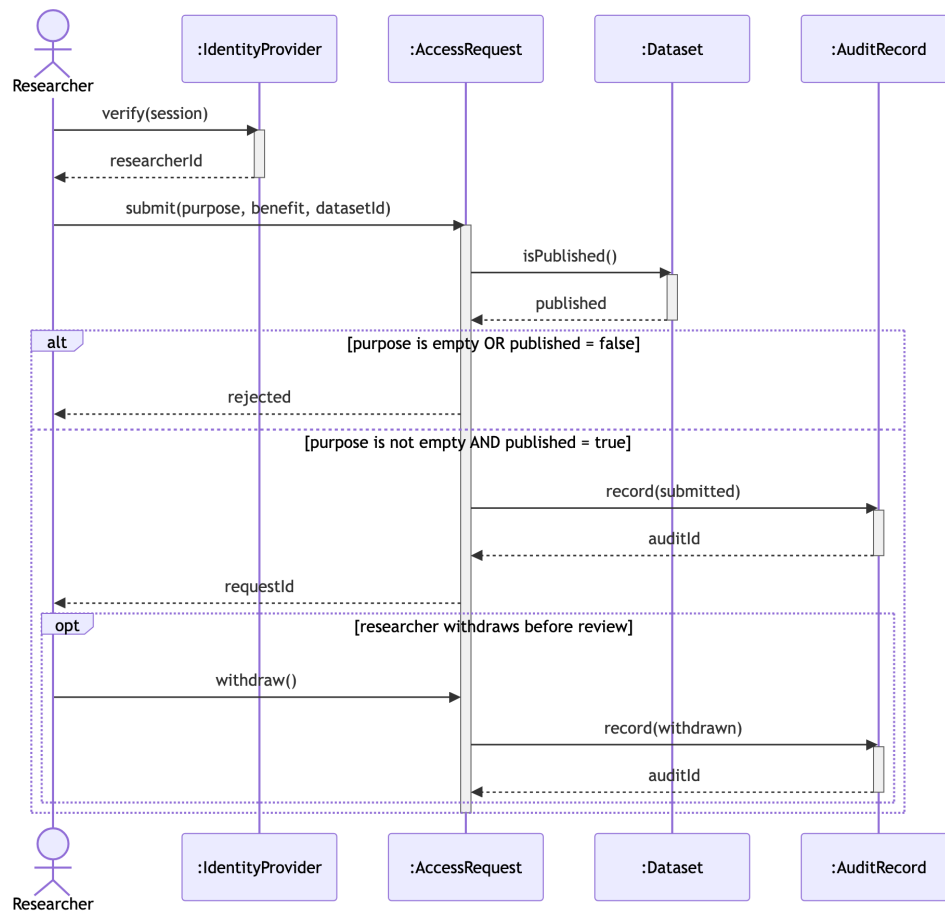
Row	Detail
Response	The status becomes Approved or Denied. AuditRecord stores that decision with the custodian and the time. NotificationService.notify tells the researcher. On approval, the researcher can take the accepted analysis file.
Pre/post-conditions	Pre: the custodian has a session, and the request is Submitted. Post: the decision, the actor, and the time are stored, and the researcher has been notified.
Comments	Supports PP-4. This is the second half of the class walkthrough. Only a user in the Custodian role can approve, deny, or revoke.
Alternative flows	1. The status is already Withdrawn: AccessRequest returns closed, and no decision is stored. 2. The custodian denies the request but leaves the reason empty: AccessRequest returns rejected, and the status stays InReview.

A4. Sequence diagrams

Lifelines use the class names from A5. A solid arrow with a filled head is a synchronous call. An open arrowhead is asynchronous. A dashed arrow is a return. Guards are in brackets.

UC-6 Request dataset access

Caption: This sequence supports PP-4 and matches the UC-6 steps, including the rejection and withdrawal paths.

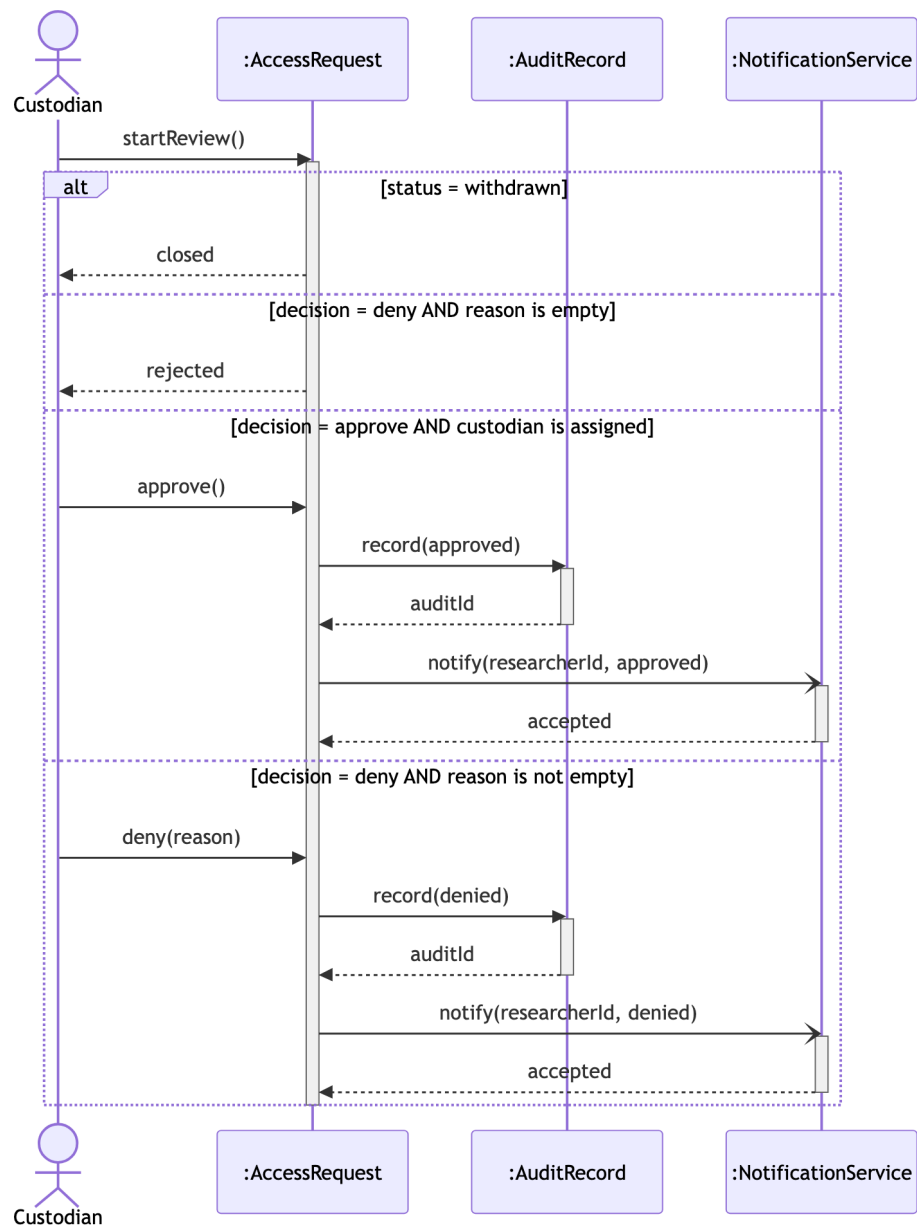


Sequence for UC-6

Identity provider is the external system from the context diagram. The alt fragment is the failure path from UC-6. The opt fragment is the cancellation path.

UC-7 Review access request

Caption: This sequence supports PP-4 and matches the UC-7 steps, including a withdrawn request and a denial with no reason.

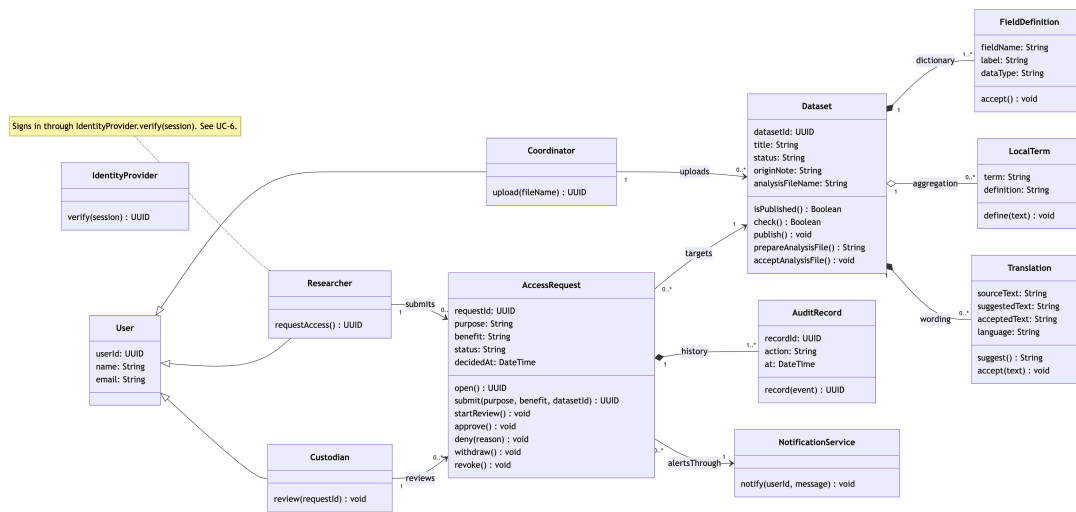


Sequence for UC-7

Notification service is the external system from the context diagram. The notify messages use an open arrowhead because the TRE does not wait for the message to be delivered. The returns are dashed.

A5. Domain class diagram

Caption: This class diagram supports PP-1, PP-2, PP-3, PP-4, and PP-5. Dataset carries the dictionary, local terms, and translations. AccessRequest carries the approval record.



Class diagram

Researcher, Coordinator, and Custodian are kinds of User. That is the generalization. A person has one role in this MVP, and only Custodian has review, approve, deny, and revoke.

FieldDefinition is a **composition** inside Dataset. A dictionary row names one column of that dataset and has no use if the dataset is deleted. AuditRecord is a composition inside AccessRequest for the same reason: the history belongs to that request. Translation is a composition because the accepted wording is part of that dataset's labels.

LocalTerm is an **aggregation**, drawn with a hollow diamond on Dataset. A hospital can reuse one term definition on a later dataset, so deleting one dataset does not delete the term. Researcher calls IdentityProvider.verify(session). That call is the note on Researcher and the first message in the UC-6 sequence.

IdentityProvider and NotificationService are the external systems from A1. They are on this diagram because the sequence diagrams call them.

Every message in A4 is an operation on the class that receives it:

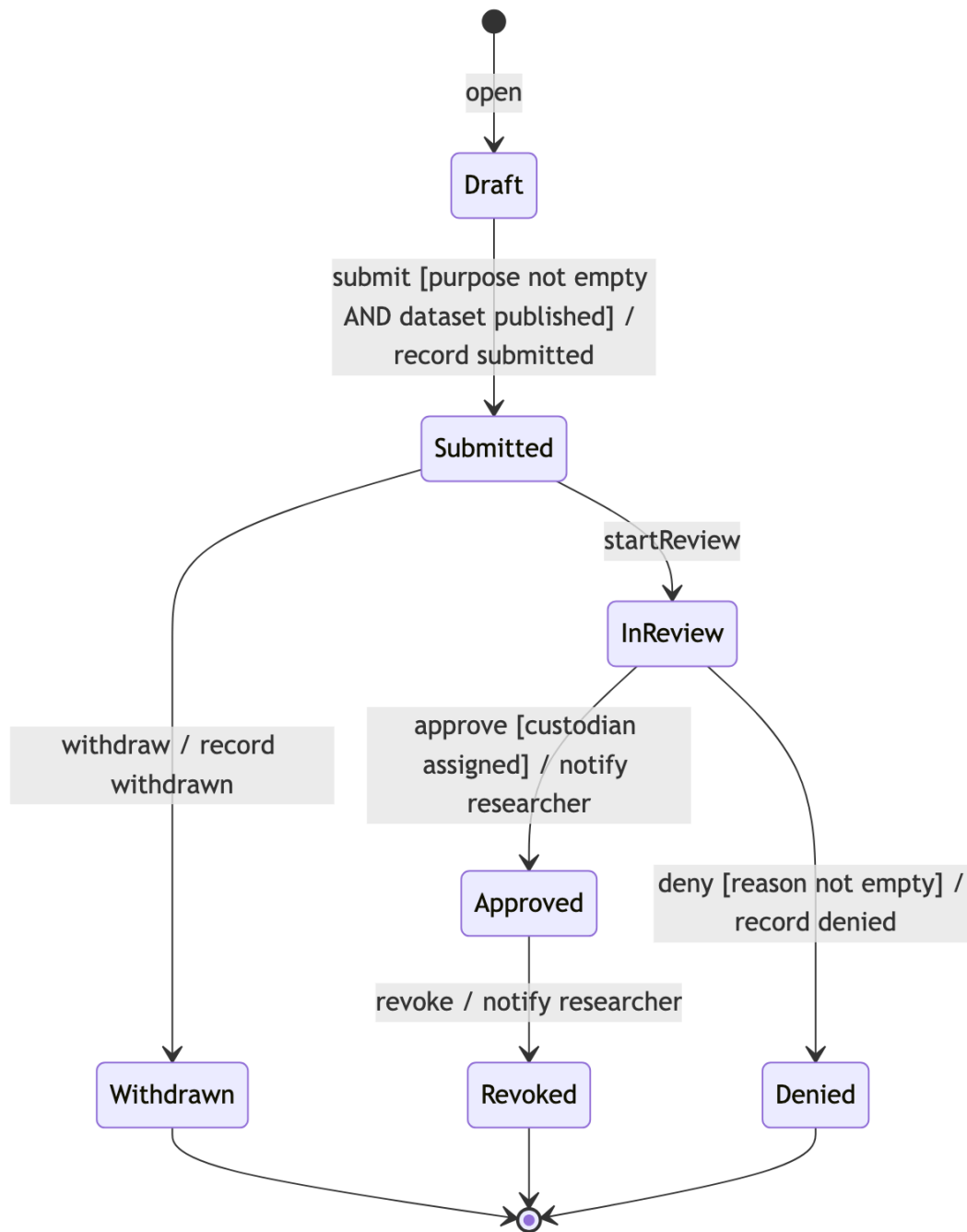
Sequence message	Operation
verify(session)	IdentityProvider.verify(session)
submit(purpose, benefit, datasetId)	AccessRequest.submit(purpose, benefit, datasetId)
isPublished()	Dataset.isPublished()
record(submitted), record(withdrawn), record(approved), record(denied)	AuditRecord.record(event)
withdraw()	AccessRequest.withdraw()
startReview()	AccessRequest.startReview()
approve()	AccessRequest.approve()
deny(reason)	AccessRequest.deny(reason)

Sequence message	Operation
notify(researcherId, approved) and notify(researcherId, denied)	NotificationService.notify(userId, message)

UC-1 through UC-5 use the other operations: Coordinator.upload, Dataset.check, Dataset.publish, LocalTerm.define, Dataset.prepareAnalysisFile, Dataset.acceptAnalysisFile, and Translation.suggest plus Translation.accept. AccessRequest.open and AccessRequest.revoke are used by the activity diagram and the state machine.

A6. State machine diagram

Caption: This state machine supports PP-4. The entity is AccessRequest, the record Daniel Okoro uses to control who receives an analysis file.



AccessRequest state machine

There are seven states: Draft, Submitted, InReview, Approved, Denied, Withdrawn, and Revoked. Denied, Withdrawn, and Revoked each reach a final state.

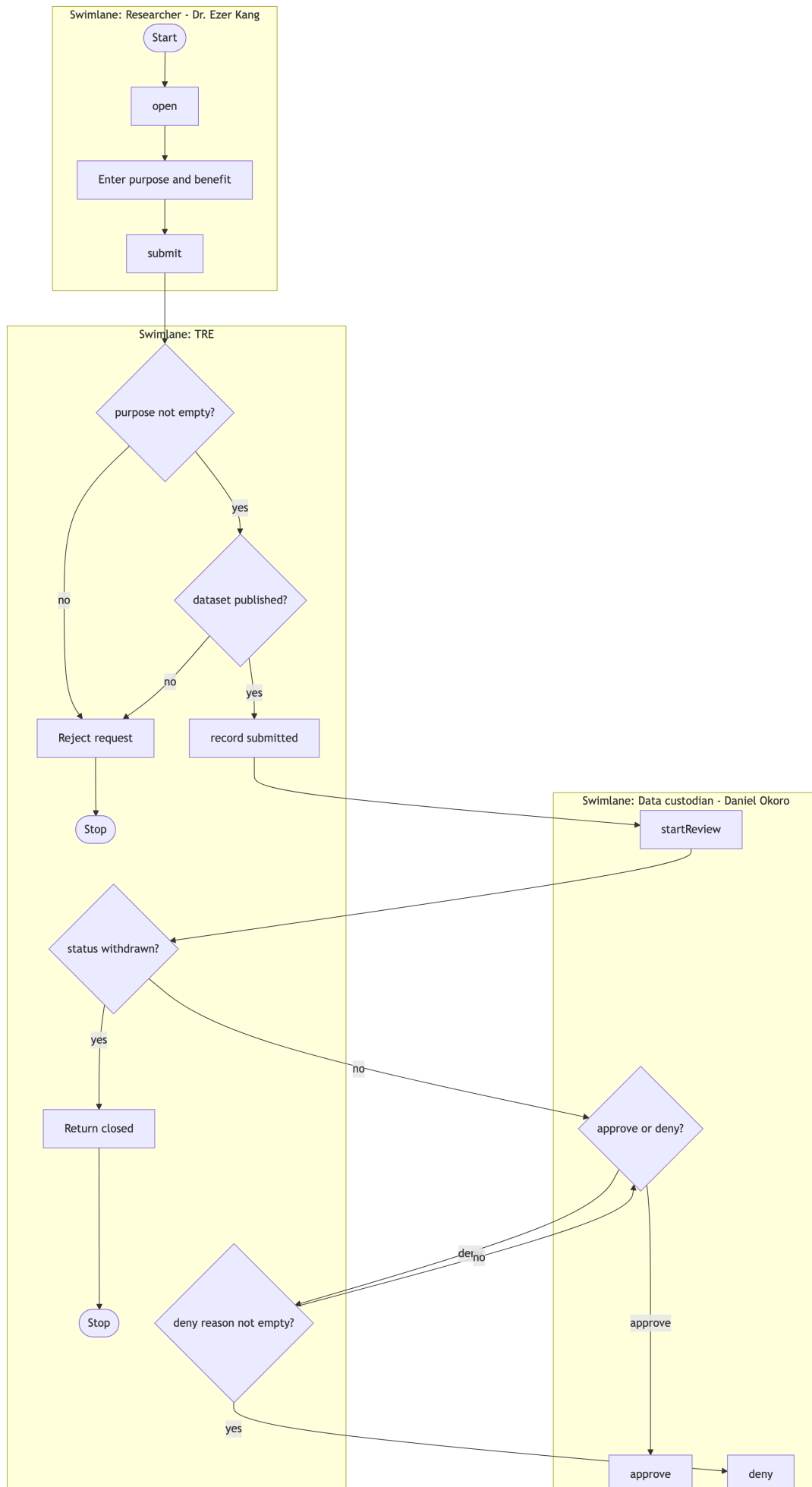
The submit transition carries a guard and an action. Approval is refused when no custodian is assigned. Denial is refused when the reason is empty. Withdraw is the cancellation path. Deny is the failure path. Revoke is how the custodian takes back a file that was already released.

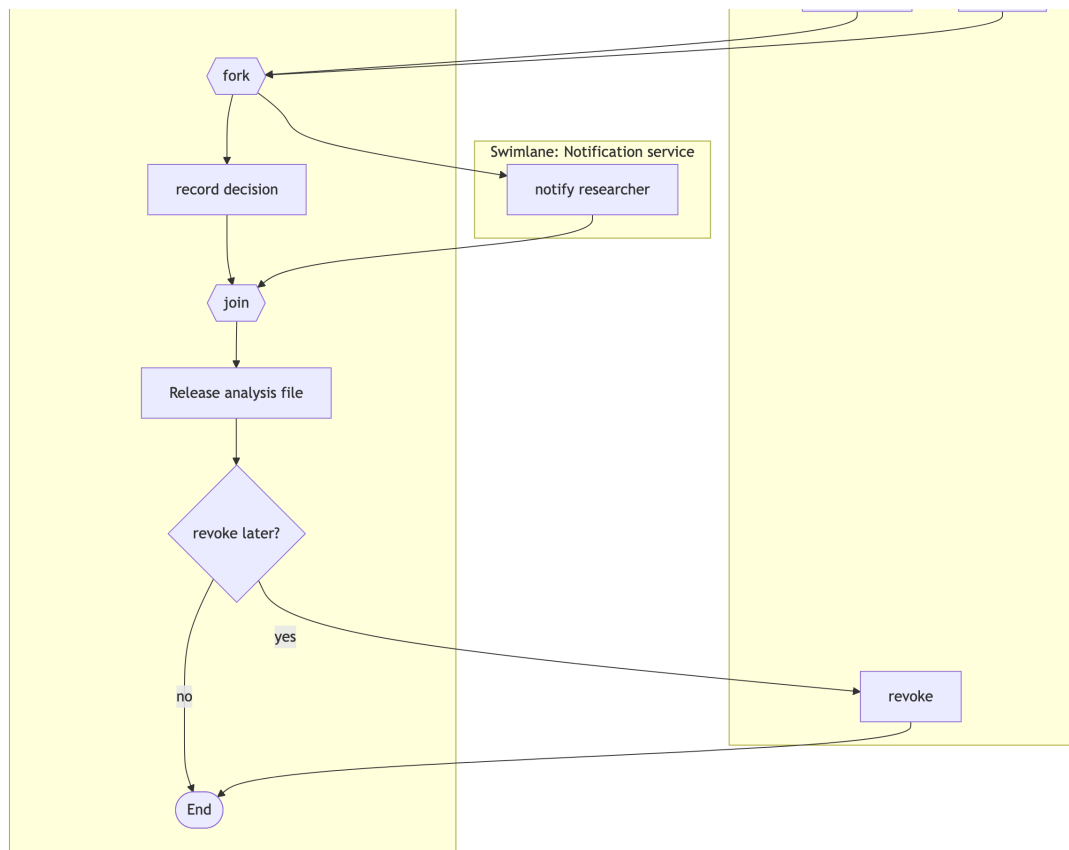
Each event also appears in a sequence diagram or in the activity diagram:

Event	Where it appears
open	A7 activity, researcher lane
submit	A4 sequence UC-6, and A7
withdraw	A4 sequence UC-6, opt fragment
startReview	A4 sequence UC-7, and A7
approve	A4 sequence UC-7, and A7
deny	A4 sequence UC-7, and A7
revoke	A7 activity, custodian lane

A7. Activity diagram

Caption: This activity diagram supports PP-4, the pain point the walkthrough traces: a researcher asks for a published dataset, and the custodian decision is recorded before any analysis file is released.





Activity diagram

The swimlanes are the researcher persona, the TRE, the custodian persona, and the notification service from the context diagram.

The decision diamonds are guarded. An empty purpose or an unpublished dataset stops the flow. A denial with an empty reason returns to the custodian. Approve and deny then fork. Recording the audit decision and notifying the researcher run in parallel, then join. The analysis file is released only after that join. Revoke is the later path from a released file back to the custodian, matching the Approved to Revoked transition.

B1. Architecture decision record

Caption: This decision supports PP-1, PP-2, PP-3, PP-4, and PP-5. The driving pain point is PP-4, because a custodian decision has to be stored with the same dataset, dictionary, and translation the other pain points create.

1. Context

Four people are building the first version. The users are a researcher, a clinical coordinator, and a data custodian. Load is a class-project load: uploads during an active study, and access requests a few times a week, not a national surveillance feed.

The data is sensitive health data. The source file and the dictionary must stay together, because a released analysis file with no local-term definitions would recreate PP-3. A translation must not be published until Dr. Kang accepts it (PP-5). The approval in PP-4 has to be one transaction with the audit record. If the decision were saved and the audit write failed, Daniel Okoro would not be able to answer who received the file.

2. Decision

We will build a **modular monolith**. One application, one deployable, one database. Inside it, four modules match the use cases:

Module	Use cases	What it owns
Intake	UC-1, UC-2, UC-3	Upload, check report, local terms
Dictionary	UC-4	Field definitions and the analysis file
Translation review	UC-5	Suggested text and the accepted text
Access	UC-6, UC-7	AccessRequest and AuditRecord

The modules call each other in process. The Kubernetes manifest in Part C deploys this one application.

3. Alternatives considered

Microservices. Intake, dictionary, translation review, and access would be four services. We rejected that. The team is four students, and the approval path needs the dataset status, the accepted analysis file, and the audit row at the same time. Splitting them puts a network call and a partial-failure case in the middle of PP-4. We would also operate four images before we have a second real user group.

Event-driven. A broker would carry “dataset published” and “request decided” events. We rejected that as the core style. The coordinator and the custodian are waiting on a form. They need a direct answer, not a later event. The one place we still want a non-blocking call is notification, and that is a single outbound call from the access module. A broker for every step would add a component the pain points do not need.

Serverless. Each use case would be a function. We rejected that. File checks and dictionary drafts run longer than a short request, and the approval rules would be scattered across functions. A long-running application process is easier for this team to trace from the class diagram to the manifest.

4. Performance consequences

Latency on the approval path is the database round trip inside the monolith, plus the identity check at the start of UC-6. There is no service-to-service hop between

AccessRequest and Dataset. The slow calls are the ones that leave the process: object storage while Dr. Kamau uploads, and the translation service while Dr. Kang asks for a suggestion. Those calls sit behind UC-1 and UC-5, not behind the custodian's decision.

The application scales by running more copies of the same image. The Part C manifest starts at two replicas and adds pods when CPU stays high. The database is the shared bottleneck. Published status is a small read on every request, so the access module may cache isPublished for a dataset until publish() changes it. Notification is asynchronous, which is the open arrow on the UC-7 sequence. approve() returns after the audit row is stored. It does not wait for email delivery.

5. Security consequences

Trust boundaries are the user network, the application, the health-data stores, and the external partners. The picture is B2. Every crossing is HTTPS, except the database, which is SQL over TLS.

Authentication is the identity provider. IdentityProvider.verify(session) is the first call in UC-6. The TRE does not store passwords. Authorization is the User subtype. Coordinator can upload. Researcher can request access and accept a dictionary or a translation. Only Custodian can startReview, approve, deny, and revoke.

Secrets are the database password and the translation API key. They come from a Kubernetes Secret. The manifest points at that Secret and does not contain the values. Source uploads live in object storage. The database holds the dictionary, terms, accepted translations, requests, and audit rows. Approval releases the analysis file only. The source upload stays in storage. That is the PP-4 control.

6. Cloud-agnostic plan

Capability	Open standard	Lock-in we accept
Run the application	Kubernetes Deployment and Service	None in the manifest. A managed Kubernetes control plane is replaceable with another conformant cluster.
Sign-in	OpenID Connect	We accept whichever identity provider speaks OIDC.
File storage	S3-compatible API	We do not depend on one vendor's storage features beyond put and get.
Records	SQL, PostgreSQL wire protocol	The schema is ordinary SQL.
Suggested translation	HTTPS JSON	We accept a translation API. The accepted text is stored in our

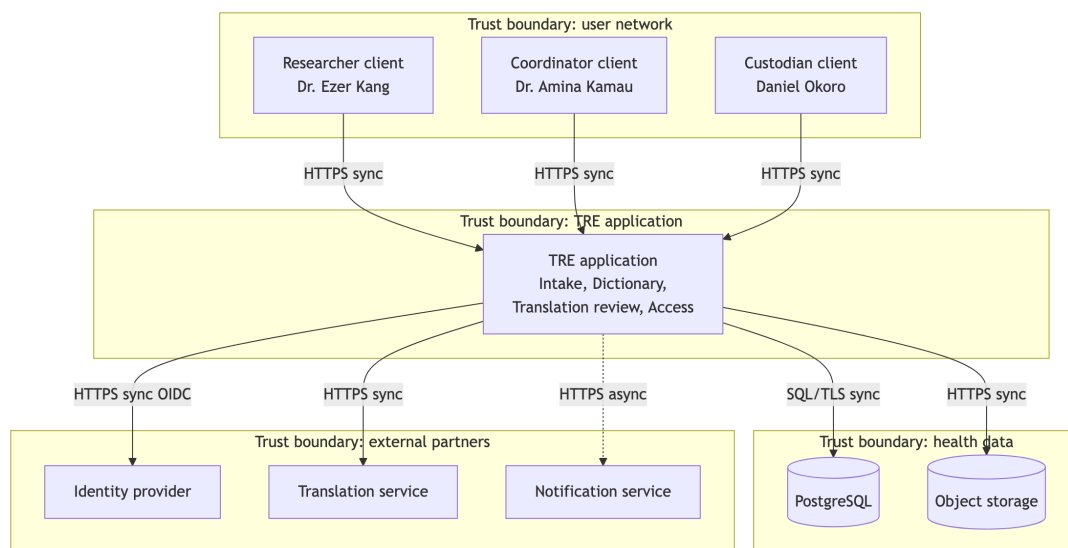
Capability	Open standard	Lock-in we accept
		database, so a provider can be swapped.
Notification	HTTPS JSON	Delivery is external. The audit record does not depend on it.

We do not accept lock-in for the approval rules. Those stay in the access module, which we build.

B2. Architecture diagram

Caption: This architecture supports PP-1, PP-2, PP-3, PP-4, and PP-5. The TRE application is the modular monolith from B1. Arrows that cross a trust boundary are labeled with the protocol.

Legend, matching the lecture: a box is a client, a service, or an external system. A cylinder is a data store. A dashed box is a trust boundary. A solid arrow is synchronous. A dotted arrow is asynchronous. There is no message broker. B1 rejected an event-driven core. Notification is a direct async call.



Architecture diagram

The modules inside the application match the class clusters. Intake and Dictionary own Dataset, FieldDefinition, and LocalTerm. Translation review owns Translation. Access owns AccessRequest and AuditRecord. Identity provider and Notification service are the same external systems as on the context diagram and the class diagram.

Each arrow that leaves a dashed box is labeled. Authentication at the identity provider is the verify(session) call in the UC-6 sequence. The security story for these crossings is in B1 section 5.

C. Kubernetes manifest

Caption: This manifest supports PP-4 and the rest of the MVP by deploying the one TRE application from B2. The access module, the dictionary, and translation review run in that same application.

A Kubernetes manifest is a YAML file that states the desired setup: which image to run, how many copies, how much CPU and memory each copy gets, and how other programs reach it. Kubernetes keeps the cluster in line with that file. This project does not need application code or a container build. The manifest points at a made-up image, `tre-team/app:0.1.0`.

The files are `deploy/tre-app.yaml`. `kubeconform` checks them with no cluster.

How it scales and how it is secured

The Deployment starts at 2 replicas, so one pod can restart while the other still answers. The HorizontalPodAutoscaler raises that count when average CPU stays above 70 percent, and it stops at 6 replicas. The Service type is ClusterIP, so only traffic from inside the cluster can reach the pods. The NetworkPolicy allows inbound traffic only to port 8080 and outbound traffic only to DNS, to PostgreSQL on 5432, and to HTTPS on 443 for the identity provider, object storage, translation, and notification. The database password and the translation API key are read from the Secret named `tre-db` and the Secret named `tre-api`. The manifest does not contain those values. `runAsNonRoot: true` keeps the process off the root user.

```
$ kubeconform -verbose -summary -strict deploy/tre-app.yaml
deploy/tre-app.yaml - Service tre-app is valid
deploy/tre-app.yaml - HorizontalPodAutoscaler tre-app is valid
deploy/tre-app.yaml - NetworkPolicy tre-app is valid
deploy/tre-app.yaml - Deployment tre-app is valid
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
exit code: 0
```

kubeconform result

Contribution statement

Proposed leads for the submission table. Confirm or edit this before the group submits. Every member is assigned at least one diagram.

Member	Lead deliverables
Anu	A1 Context diagram, A2 Use case diagram
Ade	A3 Textual use cases, A4 Sequence diagrams
Manushi	A5 Class diagram, A6 State machine

Member	Lead deliverables
Jibek	A7 Activity diagram, B2 Architecture diagram

Scope, the architecture decision record, and the Kubernetes manifest were drafted for the whole group. Put the person who will speak for each of those on this table before the walkthrough.